

CMSE 801 - Day 8

Modeling with ODEs

Sep 28, 2022

General Course Announcements

- Pre-class assignments for Monday are now due Saturday at 5:30 p.m. instead of Friday
- Pre-class assignments for Wednesday are now due Tuesday at 5:30 p.m.
- Homework 1 has been released and is due Wednesday Sep 28 at 11:59 p.m.
- Homework 2 has been released Tuesday Sep 27 and is due Wednesday Oct 12 at 11:59 p.m.

Questions/comments from Day 8 PCA

- we should go over the state space for people unfamiliar with it. I would personally appreciate a more difficult ode to walk through.
- I have a problem understanding how the parameters in odeint translate the actual differential equation.... specially the function.
- I received an error when importing Scipy that the module was not found.
- Why do we have to define a function for the derivative? Is it just for a clean code or it is really necessary?
- a lot of questions - I thought I was starting to get the material, but now I'm very confused. Maybe explain the logic of derivatives again? / Just what my errors may have been
- maybe go over more examples in class. / please give us more example about different questions
- Can we please go over all the problems in this assignment again?
- I just feel that the derives function above is totally useless procedure if we do not use ODIENT.
- Project Idea / Can you talk more about semester project? Like the topics and requirement.
- Are the semester projects an individual or group assignment? Will there be any opportunity to collaborate with other students on a project?
- just enjoying the pace and level of difficulty / Thanks for your enthusiasm on the subjects we are learning! It makes me excited to learn them / Still enjoying it a lot even though it's hard / Thank you!

Initial value problem

$$y'(t) = -5y(t)$$

$$y(0) = 1.1$$

We want an estimation of $y(3)$.

$$y'(t_n) \approx \frac{y(t_{n+1}) - y(t_n)}{t_{n+1} - t_n} = \frac{y(t_n + \Delta t) - y(t_n)}{\Delta t}$$

$$\frac{y_{n+1} - y_n}{\Delta t} = -5y_n$$

$$y_{n+1} = y_n - 5\Delta t y_n$$

$$y_{\text{new}} = y_{\text{old}} - 5\Delta t y_{\text{old}}$$

If we know the initial condition $y_0 = 1.1$ (or $y_{\text{old}} = 1.1$), we can find an estimation of $y(3)$!

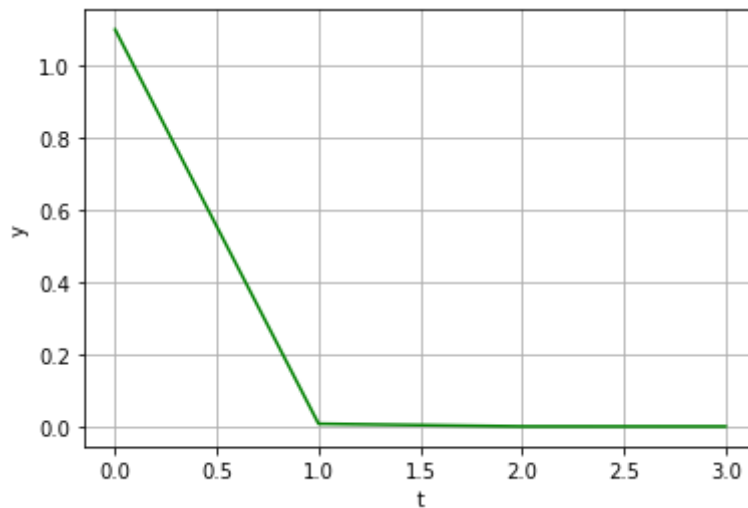
```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# Derivative function
def derivs(curr_vals, time):
    dydt = -5*curr_vals
    return dydt
```

```
In [2]: # initial condition
y0 = 1.1
# Define the time array
t0 = 0.0
tmax = 3.0
dt = 1
time = np.arange(t0, tmax + dt, dt)

# Solve the odes with odeint
sol = odeint(derivs, y0, time)
```

```
In [3]: plt.figure()
plt.plot(time, sol,color="green")
plt.xlabel('t')
plt.ylabel('y')
plt.grid()
```



```
In [4]: # pre-class examples

# Derivative function
def derivs(curr_vals, time):

    # Declare parameters
    g = -9.81 # m/s^2

    # Unpack the current values of the variables we wish
    # to "update" from the curr_vals list
    h, v = curr_vals

    # Right-hand side of odes, which are used to computer the derivative
    dhdt = v
    dvdt = g

    return dhdt, dvdt
```

```
In [5]: # Declare Variables for initial conditions
h0 = 3000 # meters
v0 = 0 # m/s
tmax = 10 # seconds
dt = 1 # seconds

# Define the time array
time = np.arange(0, tmax + dt, dt)

# Store the initial values in a list
# WARNING: it must be consistent with the
#         function derivs
init = [h0, v0]

# Solve the odes with odeint
sol = odeint(derivs, init, time)
```

```
In [6]: plt.figure
# Plot the height using the "0" element from the solution
plt.plot(time, sol[:,0],color="green")
# Plot the velocity using the "1" element from the solution
plt.plot(time, sol[:,1],color="purple")
plt.xlabel('Time [s]')
plt.grid()
```

